

Test Driven Development

Introducción

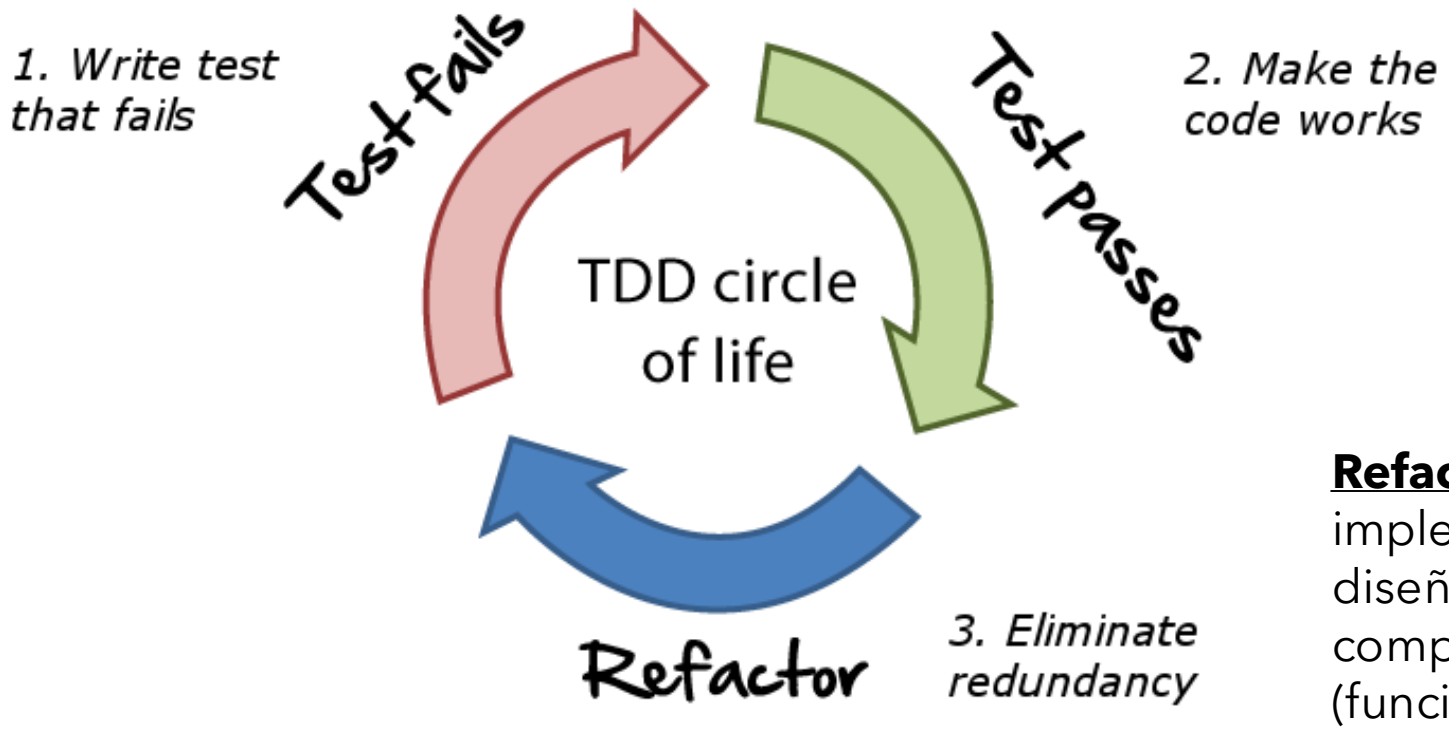
- El Desarrollo Dirigido por Pruebas (TDD) es un enfoque para el desarrollo de programas en el que se entrelazan las pruebas y el desarrollo de código.
- Las pruebas se escriben **antes** de que se escriba código y el hecho de 'pasar' las pruebas es el motor fundamental del desarrollo.
- Se desarrolla código incremental, junto con una prueba para ese incremento. No se debe pasar al siguiente incremento hasta que el código pase su prueba.
- TDD fue presentado como parte de los métodos ágiles como Extreme Programming (XP).

Tipos de Testing

- **White Box Testing**
  - Su proceso se basa en el conocimiento del diseño de la función bajo prueba
  - Requiere conocimiento interno sobre el funcionamiento del programa
  - Requiere habilidades de programación
  - ***Estamos construyendo el software correctamente?***

- **Black Box Testing**
  - Testing funcional
  - No se conoce o no es necesario conocer aspectos internos del programa / función
  - No requiere habilidades de programación
  - El Tester sabe lo que programa hace pero no sabe cómo lo hace
  - ***Estamos construyendo el software correcto?***

Desarrollo Dirigido por Pruebas



Refactor: Es el cambio de la implementación (mejora del diseño) sin cambiar el comportamiento (funcionalidad)

Desarrollo Dirigido por Pruebas

- TDD es una manera de programar que promueve el buen diseño y es un proceso disciplinado que ayuda a prevenir errores de programación mediante la creación de pruebas pequeñas y automatizadas que eventualmente conforman un sistema de alarma efectivo para proteger el código de errores de regresión.
- El Refactoring es una manera disciplinada de transformar código de un estado o estructura a otro estado, removiendo la duplicación de código y gradualmente moviendo el código hacia un mejor diseño.

Mayor Calidad con TDD

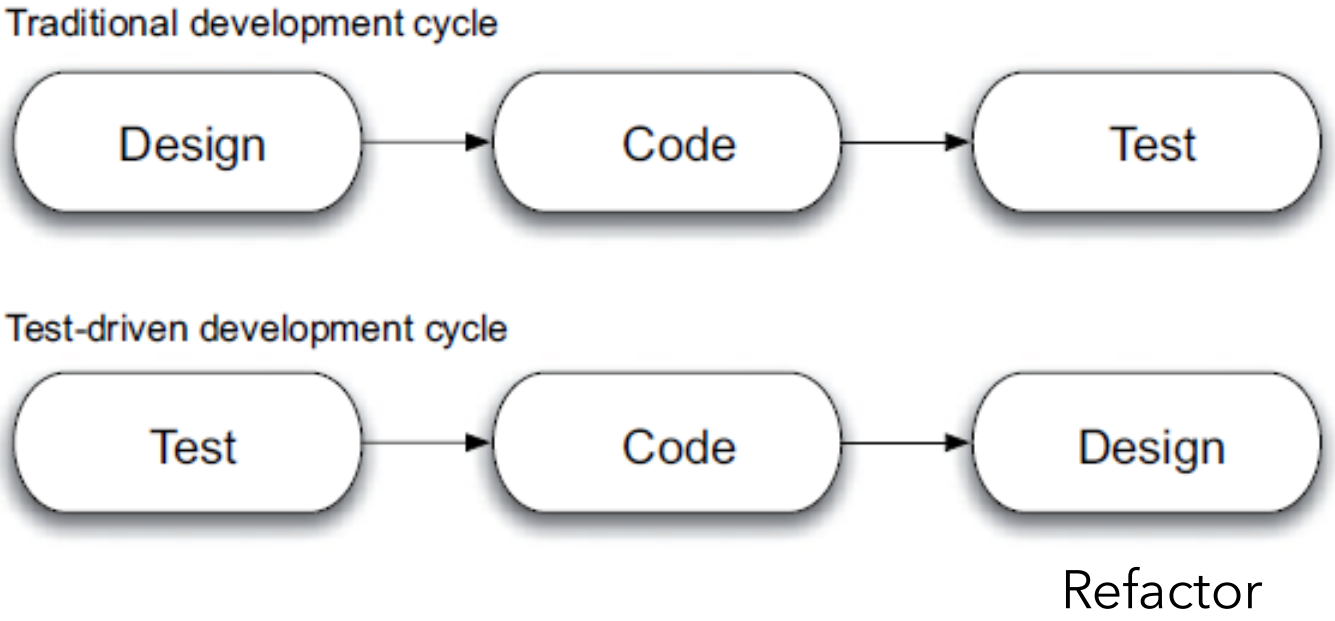
- La Calidad viene de muchas formas
- Menos tiempo empleado en corregir defectos
- No mas sesiones largas de debugging
- Sentir confianza en el programa
- Más tiempo en otras tareas

Ciclo TDD

La regla básica es:

Solo se escribe código para arreglar (fix) una prueba fallida

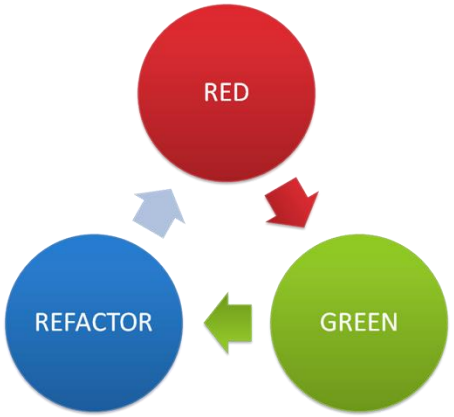
En TDD, al diseño se conoce como Refactor, entonces el ciclo es mas conocido como: *Test, Code, Refactor*.

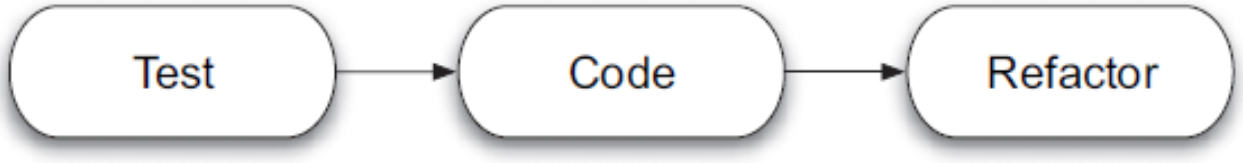


Ciclo TDD

Este ciclo tambien es conocido como: **Red-Green-Refactor.**

Esta alternativa nemotecnica surgio debido al soporte que los IDE's proporcionan a TDD.



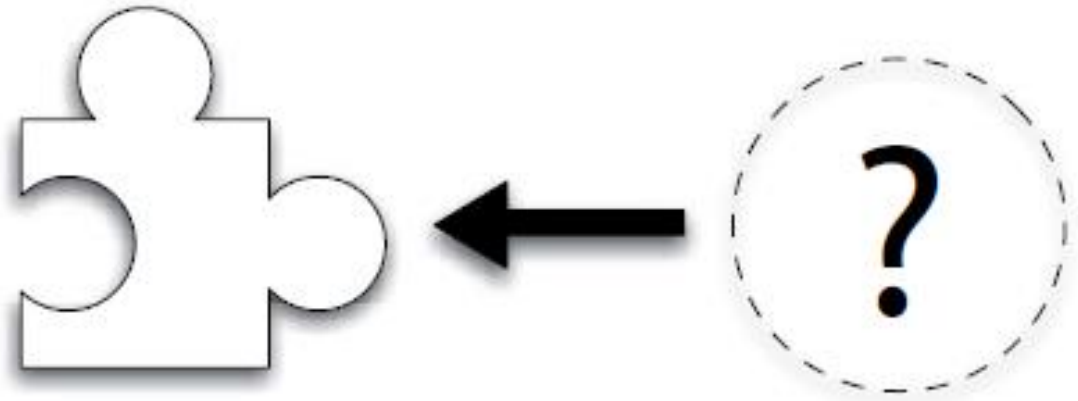


Beneficios del TDD

- ✓ Cobertura de código
  - ✓ Cada segmento de código que se escribe tiene por lo menos una prueba asociada por lo que todo el código escrito tiene por lo menos una prueba.
- ✓ Pruebas de regresión
  - ✓ Se desarrolla incrementalmente un conjunto de pruebas de regresión que se ejecutan en el programa o sistema de forma frecuente.
- ✓ Depuración simplificada
  - ✓ Cuando falla una prueba, debería ser obvio encontrar dónde radica el problema. El nuevo código tiene que ser revisado y modificado.
- ✓ Documentación del sistema
  - ✓ Las pruebas en sí mismas son una forma de documentación que describe lo que el código debería estar haciendo.

Primero se escribe una prueba

- Cuando se escribe una prueba, en realidad se están tomando decisiones de diseño.
- Se está diseñando el API, la interfaz para acceder a la funcionalidad que se está probando.
- Se debe diseñar la forma en la que se quiere que se utilice el código.



Luego se escribe solo el código suficiente

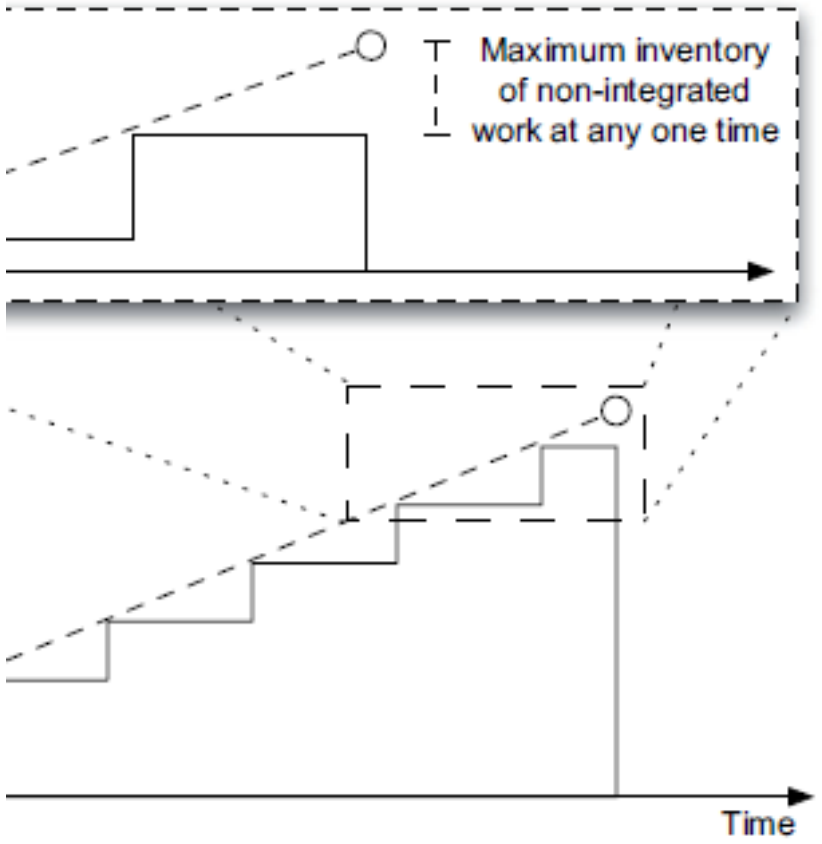
- Una de las ideas fundamentales del TDD es que permite a las pruebas mostrar qué se implementara con el objetivo de realizar progresos en el desarrollo.
- El objetivo es escribir código para hacer pasar la prueba lo más rápido posible, eso significa una implementación que **NO** es óptima.
- Por el momento esto está bien hasta que se tenga la implementación deseada y se tenga las pruebas que validen y verifiquen este comportamiento.
- Con las pruebas como una forma de verificar que se tiene la implementación deseada, se puede proceder a mejorar el diseño utilizando: **Refactoring**.

Y Luego se hace Refactoring

- El último paso del TDD, es el Refactor, este paso significa revisar el diseño y encontrar formas de hacerlo mejor.
- El Refactor hace al TDD sostenible, sin este paso, el TDD produciría código de baja calidad.
- La Calidad de código es directamente proporcional a la productividad al momento de desarrollar código.

Desarrollar en pequeños incrementos

- Desarrollar en pequeños incrementos permite que nunca estemos lejos de una base integrada (working code base).
- Esto reduce el riesgo, porque el inventario de trabajo incompleto permanece pequeño.



Herramientas, Frameworks y Ambientes

- Build
  - Gradle
  - Maven
- xUnit
  - JUnit
  - TestNG
- Hamcrest & AssertJ
- Code Coverage
  - JaCoCo: Java Code Coverage
- Mocking / Stubs
  - Mockito
  - EasyMock

TDD Kata para una Calculadora: Suma

- Requerimientos:
  - Cuando se ingrese una cadena vacia, se debe retornar cero
  - Para 2 números delimitados por coma, se debe retornar la suma
  - Para 3 números delimitados por coma, se debe retornar la suma
  - Los números mayores a 1000 deben ser ignorados
  - Para los números negativos se debe lanzar una excepción

TDD Kata para Tres en Raya

Requerimiento 1:

Una pieza puede estar colocada en un espacio de un tablero 3x3

Pruebas:

- Cuando una pieza esta fuera del eje X, entonces se debe lanzar una excepción
- Cuando una pieza esta fuera del eje Y, entonces se debe lanzar una excepción
- Cuando una pieza este en un lugar ya ocupado, se debe lanzar una excepción

TDD Kata para Tres en Raya

Requerimiento 2:

Debe existir una forma de determinar que jugador juega despues

Pruebas:

- El primer turno siempre debe ser jugado por ‘X’
- Si el ultimo turno fue jugado por ‘X', entonces el siguiente turno es para ‘+’
- Si el ultimo turno fue jugado por ‘+’, entonces el siguiente turno es para ‘X’

TDD Kata para Tres en Raya

Requerimiento 3:

Un jugador gana si es el primero en conectar una linea de un lado a otro, o de una esquina a otra

Pruebas:

- Si no se cumple la condicion de victoria, entonces no hay ganador
- Un jugador gana cuando toda una linea horizontal es ocupada por sus piezas
- Un jugador gana cuando toda una linea vertical es ocupada por sus piezas
- Un jugador gana cuando toda una linea diagonal es ocupada por sus piezas

Preguntas de Seguimiento



En grupos, durante 10 min, discutir y argumentar las siguientes preguntas:

- Que es un Assert?
- Cuales son las 3 partes de una Prueba?
- Si un metodo/funcion/codigo, no tiene pruebas unitarias, comente que se deberia hacer para hacer cambios en el codigo de forma controlada?
- Las pruebas unitarias son parte del codigo/proyecto? Explique por que?

Preguntas?

Gracias!!